

E - Commerce EDA Project

```
In [1]: import pandas as pd
import matplotlib.pyplot as plt
```

```
In [ ]: # Cleaning Products table
```

```
In [9]: products = pd.read_csv("D:\e-com data clean\products.csv")
products.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  -
0   ProductID       20 non-null    int64
1   ProductName     20 non-null    object
2   Category        20 non-null    object
3   UnitPrice       20 non-null    float64
dtypes: float64(1), int64(1), object(2)
memory usage: 772.0+ bytes
```

```
In [15]: products.isnull().sum()
```

```
Out[15]: ProductID      0
ProductName    0
Category       0
UnitPrice      0
dtype: int64
```

```
In [17]: products.duplicated().sum()
```

```
Out[17]: np.int64(0)
```

```
In [18]: products.head(5)
```

```
Out[18]:
```

	ProductID	ProductName	Category	UnitPrice
0	2001	Wireless Mouse	Electronics	18.0
1	2002	Mechanical Keyboard	Electronics	62.0
2	2003	USB-C Cable	Accessories	9.0
3	2004	Power Bank	Electronics	35.0
4	2005	Laptop Stand	Accessories	28.0

```
In [ ]: # There are no missing values or duplicate records also the data types are corre
```

```
In [ ]: # Cleaning Customers table
```

```
In [19]: customers = pd.read_csv("D:\e-com data clean\customers.csv")
customers.head(5)
```

Out[19]:

	CustomerID	Age	City	SignupDate	CustomerSegment
0	100001	22.0	Tehran	2023-09-11	Regular
1	100002	55.0	Tabriz	2024-01-16	VIP
2	100003	49.0	Karaj	2025-07-31	New
3	100004	39.0	Tehran	2023-04-23	New
4	100005	38.0	Tehran	2023-04-29	Regular

In [20]: `customers.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   CustomerID      10000 non-null  int64
1   Age             9820 non-null   float64
2   City            9881 non-null   object
3   SignupDate      10000 non-null  object
4   CustomerSegment 10000 non-null  object
dtypes: float64(1), int64(1), object(3)
memory usage: 390.8+ KB
```

In [21]: `customers.isnull().sum()`

Out[21]:

CustomerID	0
Age	180
City	119
SignupDate	0
CustomerSegment	0

dtype: int64

In [23]:

```
customers['Age'] = customers['Age'].fillna(customers['Age'].median())
customers['City'] = customers['City'].fillna('Unknown')
customers['SignupDate'] = pd.to_datetime(customers['SignupDate'])
```

In [24]: `customers.isnull().sum()`

Out[24]:

CustomerID	0
Age	0
City	0
SignupDate	0
CustomerSegment	0

dtype: int64

In [42]: `customers.duplicated().sum()`

Out[42]: `np.int64(0)`

In [25]: `customers.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  -
0   CustomerID            10000 non-null  int64
1   Age                   10000 non-null  float64
2   City                  10000 non-null  object
3   SignupDate            10000 non-null  datetime64[ns]
4   CustomerSegment       10000 non-null  object
dtypes: datetime64[ns](1), float64(1), int64(1), object(2)
memory usage: 390.8+ KB
```

```
In [ ]: # Cleaning Payments table
```

```
In [26]: payments = pd.read_csv("D:\e-com data clean\payments.csv")
payments.head(5)
```

```
Out[26]:
```

	PaymentID	OrderID	PaymentDate	PaymentStatus
0	800001	500001	2025-08-28	Paid
1	800002	500002	2024-05-31	Paid
2	800003	500003	2025-08-10	Paid
3	800004	500004	2024-10-11	Paid
4	800005	500005	2024-02-19	Paid

```
In [27]: payments.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50000 entries, 0 to 49999
Data columns (total 4 columns):
#   Column                Non-Null Count  Dtype
---  -
0   PaymentID            50000 non-null  int64
1   OrderID              50000 non-null  int64
2   PaymentDate          49965 non-null  object
3   PaymentStatus        50000 non-null  object
dtypes: int64(2), object(2)
memory usage: 1.5+ MB
```

```
In [28]: payments.isnull().sum()
```

```
Out[28]: PaymentID      0
OrderID      0
PaymentDate    35
PaymentStatus  0
dtype: int64
```

```
In [43]: products.duplicated().sum()
```

```
Out[43]: np.int64(0)
```

```
In [36]: payments['PaymentDate'] = pd.to_datetime(payments['PaymentDate'])
#35 rows out of 50,000 is only 0.07% of data so deleting them
payments = payments.dropna(subset=['PaymentDate'])
```

In [37]: `payments.info()`

```
<class 'pandas.core.frame.DataFrame'>
Index: 49965 entries, 0 to 49999
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  -
0   PaymentID       49965 non-null  int64
1   OrderID         49965 non-null  int64
2   PaymentDate     49965 non-null  datetime64[ns]
3   PaymentStatus   49965 non-null  object
dtypes: datetime64[ns](1), int64(2), object(1)
memory usage: 1.9+ MB
```

In [38]: `payments.isnull().sum()`

```
Out[38]: PaymentID      0
         OrderID      0
         PaymentDate  0
         PaymentStatus 0
         dtype: int64
```

In []: *# Cleaning Orders table*

In [71]: `orders = pd.read_csv("D:\e-com data clean\orders.csv")`
`orders.head(5)`

Out[71]:

	OrderID	CustomerID	OrderDate	ProductID	Quantity	Discount	PaymentMethod
0	500001	103695	2025-08-28	2003	4.0	0.1	Gateway
1	500002	107935	2024-05-31	2014	1.0	0.1	Wallet
2	500003	105141	2025-08-10	2002	1.0	0.2	CardToCard
3	500004	108780	2024-10-11	2012	1.0	0.1	Gateway
4	500005	101187	2024-02-19	2008	2.0	0.0	Gateway

In [72]: `orders.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50120 entries, 0 to 50119
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   OrderID         50120 non-null  int64
1   CustomerID      50120 non-null  int64
2   OrderDate       50085 non-null  object
3   ProductID       50120 non-null  int64
4   Quantity        50040 non-null  float64
5   Discount        49899 non-null  float64
6   PaymentMethod   49668 non-null  object
7   Status          50120 non-null  object
dtypes: float64(2), int64(3), object(3)
memory usage: 3.1+ MB
```

```
In [73]: orders.isnull().sum()
```

```
Out[73]: OrderID          0
CustomerID        0
OrderDate         35
ProductID         0
Quantity          80
Discount         221
PaymentMethod     452
Status            0
dtype: int64
```

```
In [74]: orders.duplicated().sum()
```

```
Out[74]: np.int64(120)
```

```
In [75]: orders = orders.drop_duplicates()
orders.duplicated().sum()
```

```
Out[75]: np.int64(0)
```

```
In [80]: orders['Discount'] = orders['Discount'].fillna(0.0)
orders['Quantity'] = orders['Quantity'].fillna(orders['Quantity'].mode()[0])
orders['PaymentMethod'] = orders['PaymentMethod'].fillna('Unknown')
orders['OrderDate'] = pd.to_datetime(orders['OrderDate'])
```

```
In [81]: orders.isnull().sum()
```

```
Out[81]: OrderID          0
CustomerID        0
OrderDate         0
ProductID         0
Quantity          0
Discount          0
PaymentMethod     0
Status            0
dtype: int64
```

```
In [82]: orders = orders.dropna(subset=['OrderDate'])
```

```
In [83]: orders.isnull().sum()
```

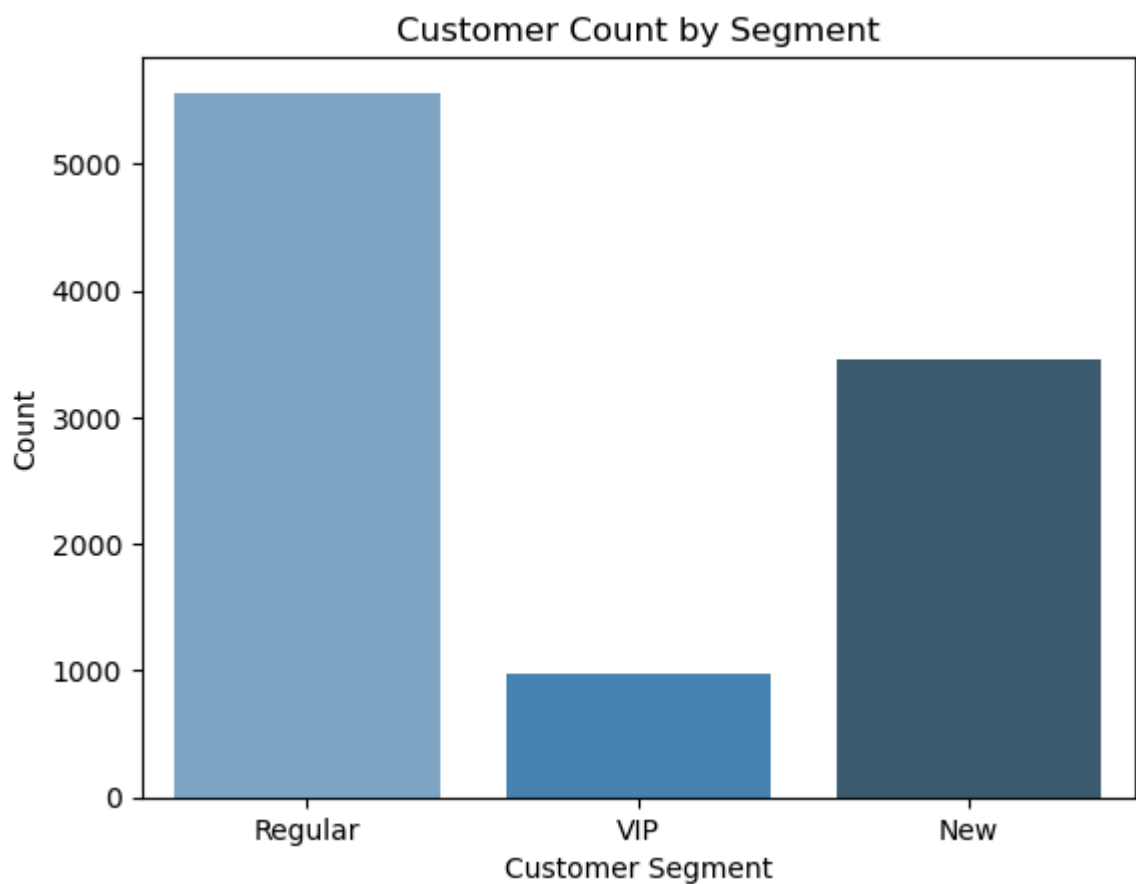
```
Out[83]: OrderID      0
CustomerID  0
OrderDate   0
ProductID   0
Quantity    0
Discount    0
PaymentMethod 0
Status      0
dtype: int64
```

EDA

```
In [ ]: # For Customers Table: Who are the users?
```

```
In [94]: import matplotlib.pyplot as plt
import seaborn as sns

#1. Customer Segment Breakdown
sns.countplot(data = customers, x = 'CustomerSegment', hue = 'CustomerSegment',
plt.title('Customer Count by Segment')
plt.xlabel('Customer Segment')
plt.ylabel('Count')
plt.show()
```

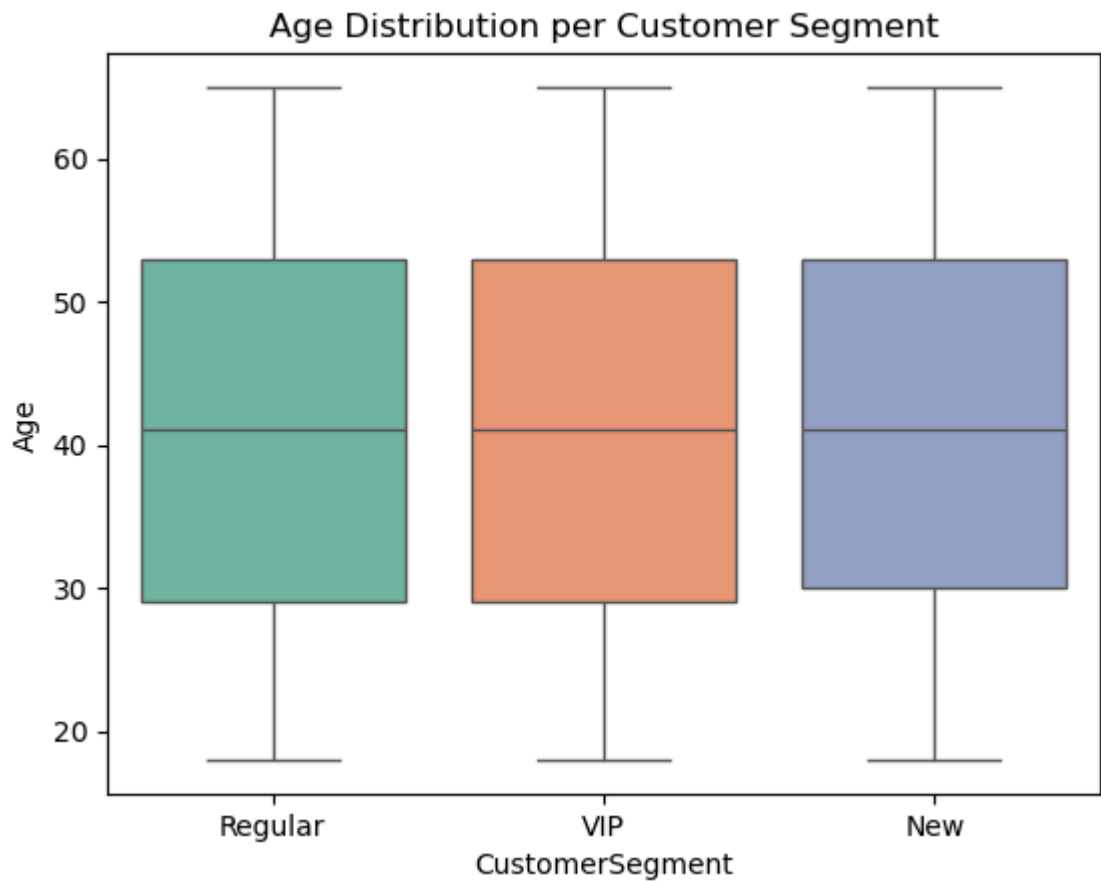


```
In [92]: #2. Top Cities
top_cities = customers['City'].value_counts().head(7)
top_cities.plot(kind = 'bar', color = 'teal')
plt.title('Top 7 Cities by Registered Customers')
plt.xlabel('City')
plt.ylabel('Number of Customers')
```

```
plt.xticks(rotation=30)  
plt.show()
```

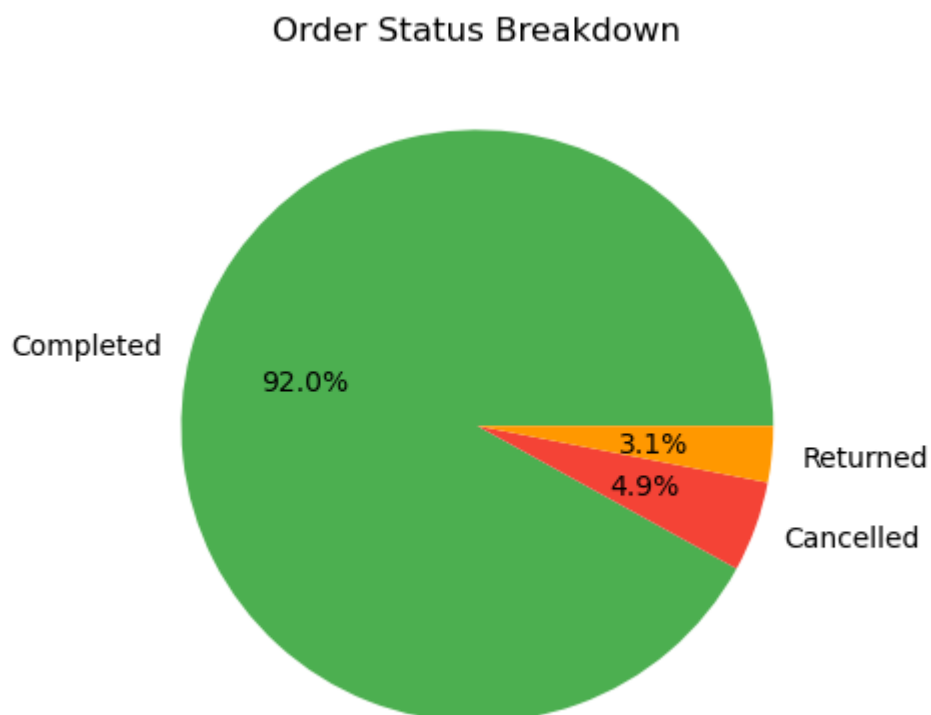


```
In [96]: #3. Age Spread Across Segments  
sns.boxplot(data = customers, x = 'CustomerSegment', hue = 'CustomerSegment', y  
plt.title('Age Distribution per Customer Segment')  
plt.show()
```



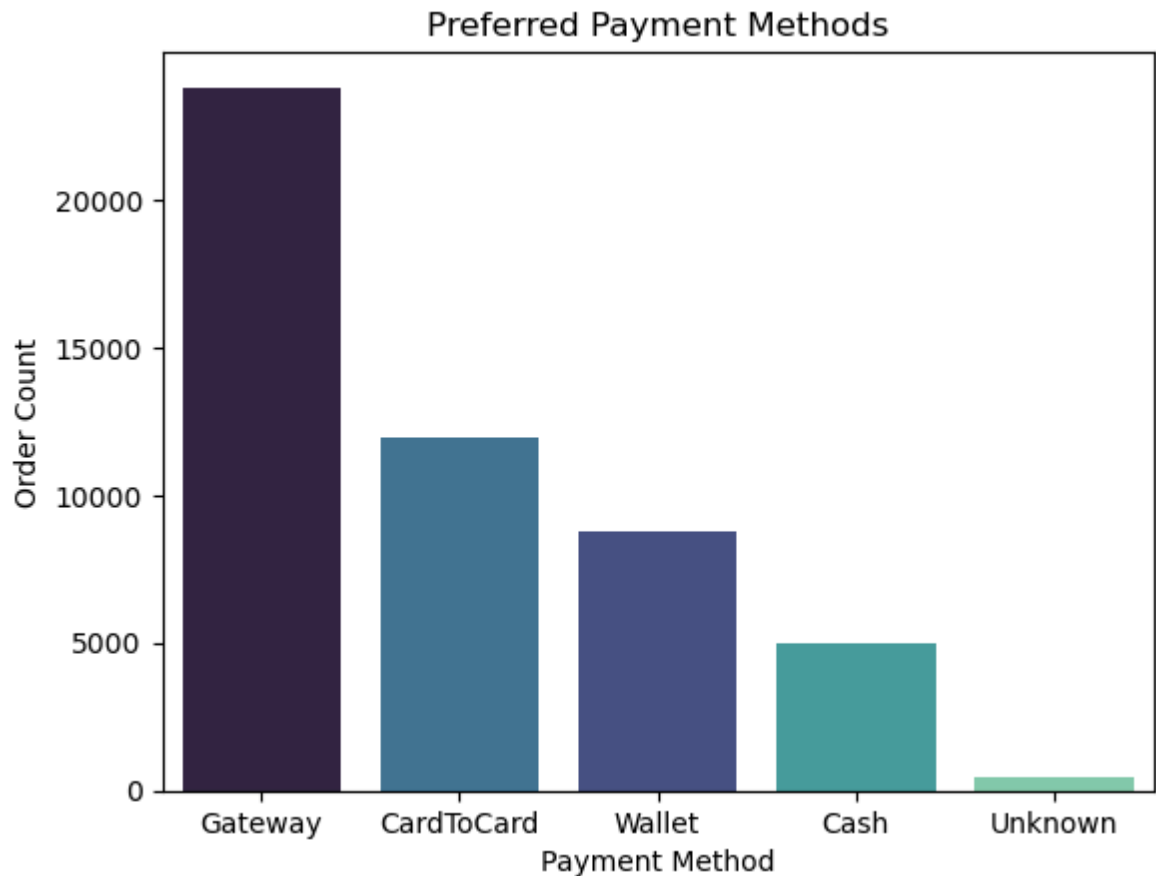
```
In [ ]: # For Orders Table: Order Patterns & Behavior
```

```
In [98]: #1. Order Status
orders['Status'].value_counts().plot(kind = 'pie', autopct = '%1.1f%%', colors = 
plt.title('Order Status Breakdown')
plt.ylabel('')
plt.show()
```



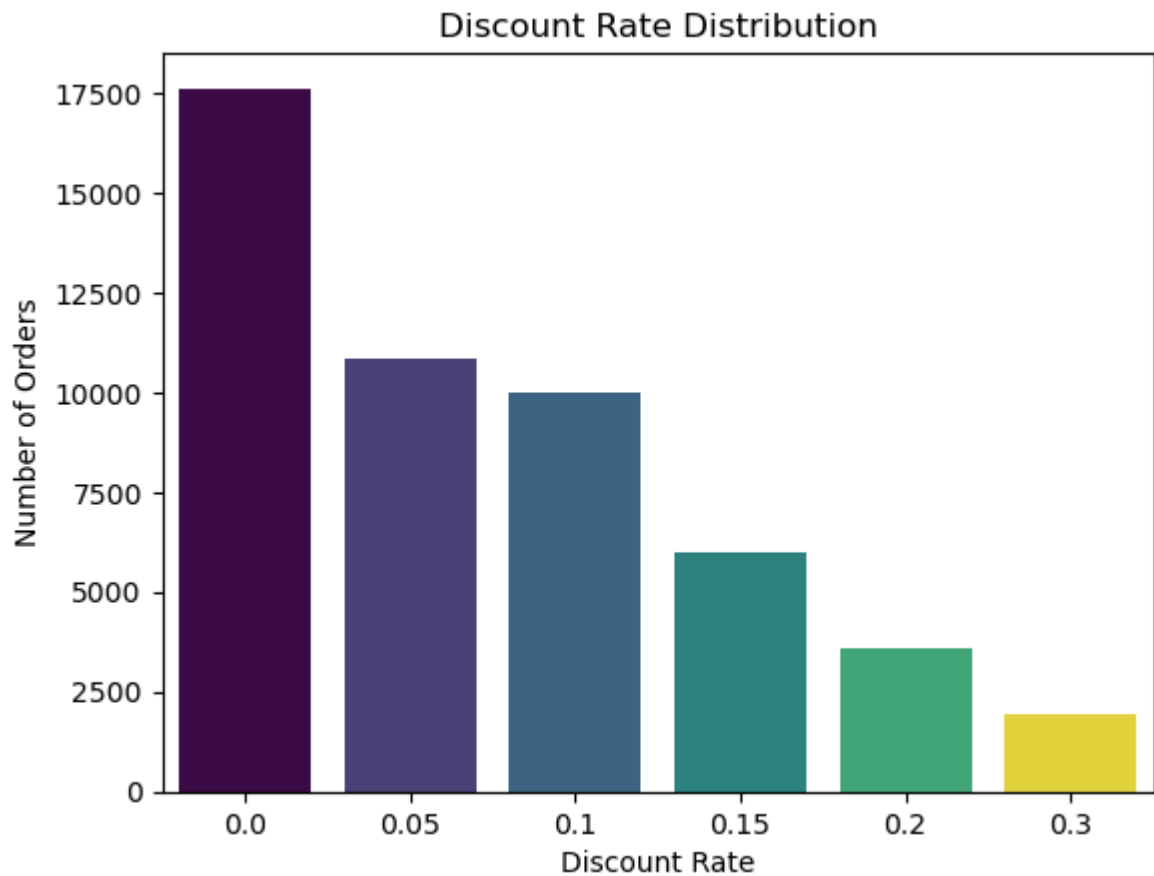
In [101...

```
#2. Most Popular Payment Channels
sns.countplot(data = orders, x = 'PaymentMethod', hue = 'PaymentMethod' , order
              legend = False)
plt.title('Preferred Payment Methods')
plt.xlabel('Payment Method')
plt.ylabel('Order Count')
plt.show()
```



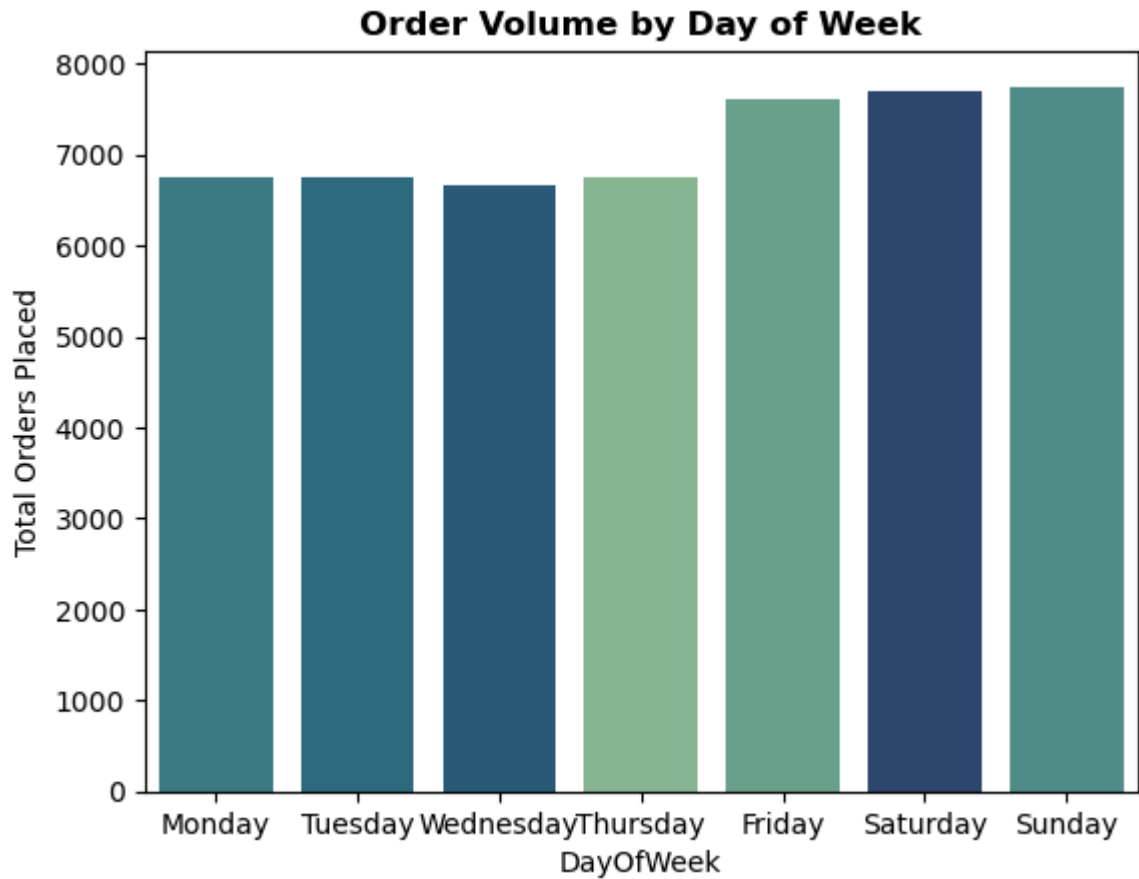
In [103...

```
#3. Discount Frequency
sns.countplot(data = orders, x = 'Discount', hue = 'Discount' , palette = 'viri
plt.title('Discount Rate Distribution')
plt.xlabel('Discount Rate')
plt.ylabel('Number of Orders')
plt.show()
```



In [110...

```
#4. Behavioral & Operational Patterns - Demand Seasonality & Day-of-Week Load
orders['OrderDate'] = pd.to_datetime(orders['OrderDate'])
orders['DayOfWeek'] = orders['OrderDate'].dt.day_name()
days_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday']
sns.countplot(data = orders, x = 'DayOfWeek', hue = 'DayOfWeek', order = days_order)
plt.title('Order Volume by Day of Week', fontsize=12, fontweight='bold')
plt.ylabel('Total Orders Placed')
plt.show()
```



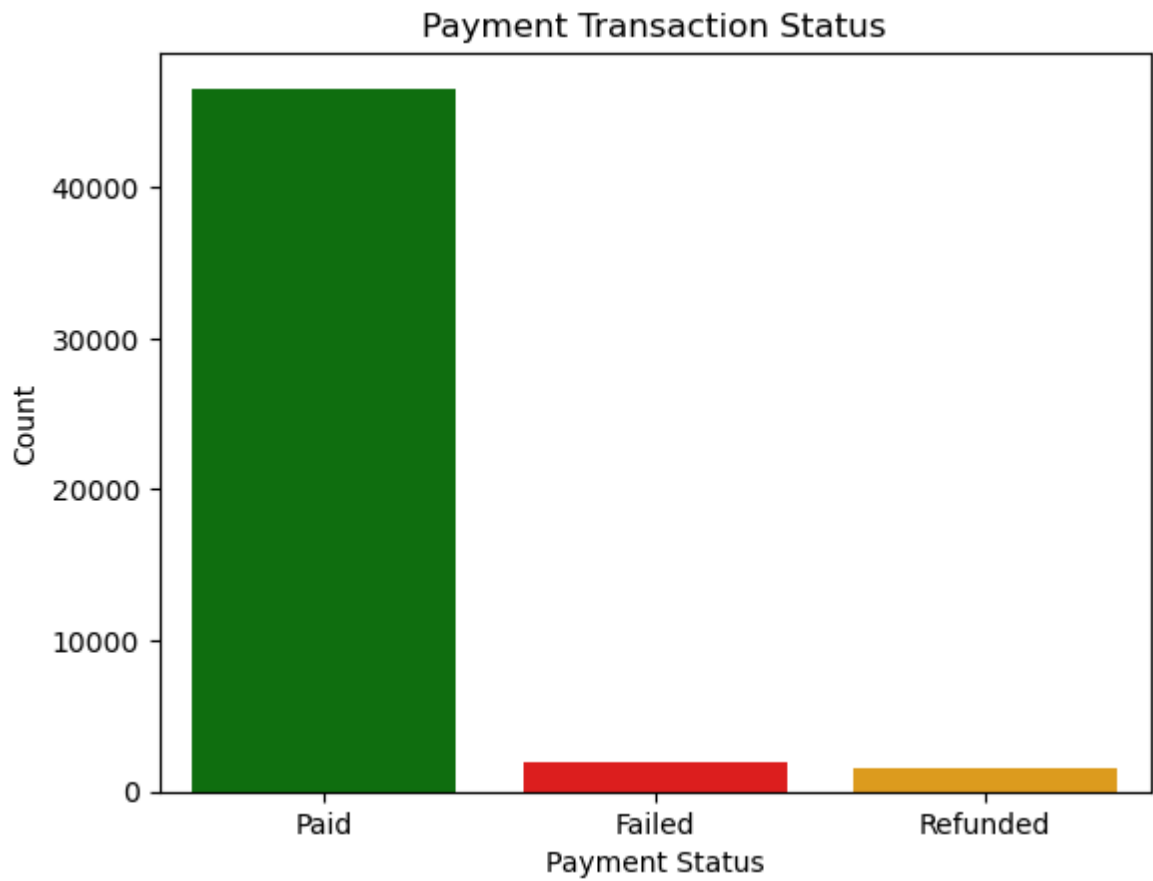
In [113...

```
#5. Discount Sensitivity vs. Order Status
# Bin discounts into Low, Medium, and High
orders['DiscountBracket'] = pd.cut(orders['Discount'], bins = [-0.01, 0.05, 0.15]
status_discount = pd.crosstab(orders['DiscountBracket'], orders['Status'], norma
print(status_discount.round(2))
```

Status	Cancelled	Completed	Returned
DiscountBracket			
Low (0-5%)	5.00	92.10	2.90
Med (10-15%)	4.97	91.64	3.39
High (20-30%)	4.60	92.44	2.96

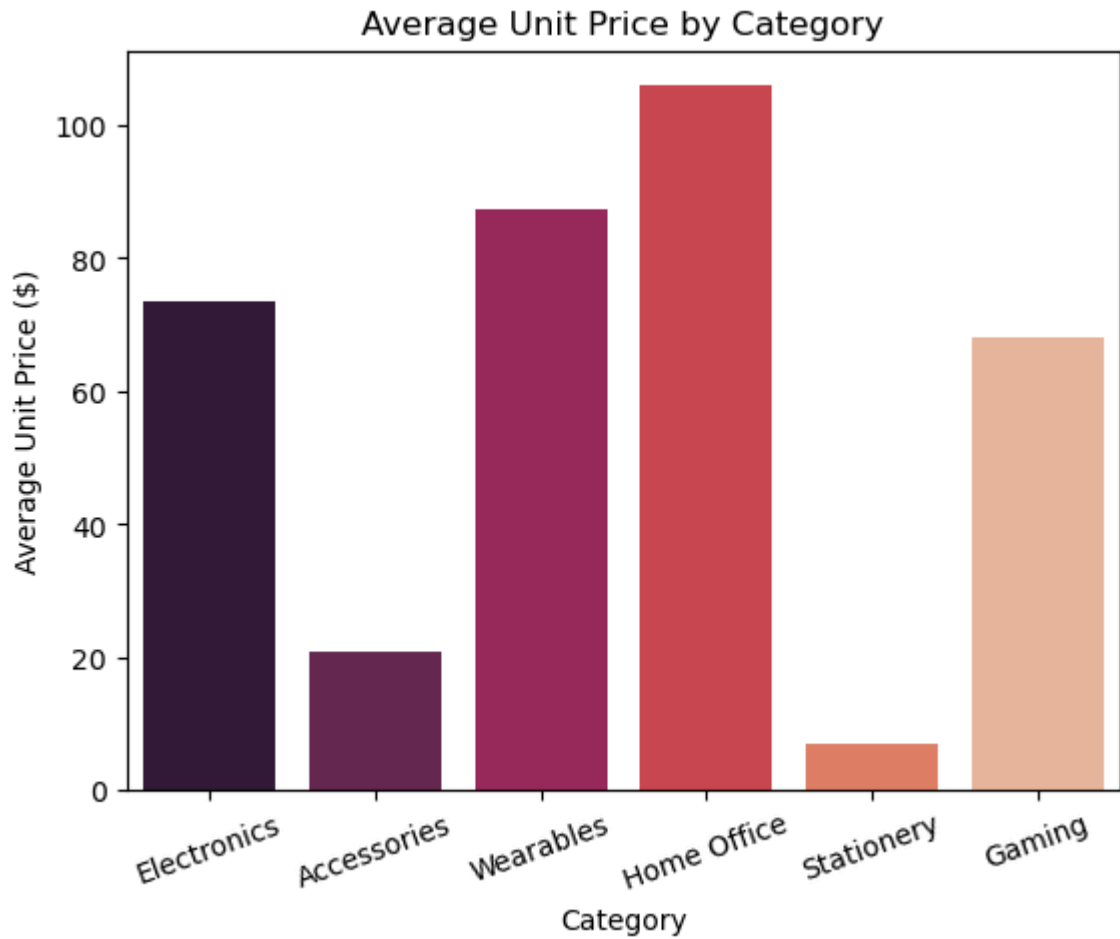
In [105...

```
# For Payments Table: Financial Health
#Payment Success vs. Failures vs. Refunds
sns.countplot(data = payments, x = 'PaymentStatus', hue = 'PaymentStatus' , pale
plt.title('Payment Transaction Status')
plt.xlabel('Payment Status')
plt.ylabel('Count')
plt.show()
```



In [108...

```
# For Products Table: Catalog & Pricing
# Price distribution across categories
sns.barplot(data = products, x = 'Category', hue = 'Category', y = 'UnitPrice', e
plt.title('Average Unit Price by Category')
plt.xlabel('Category')
plt.ylabel('Average Unit Price ($)')
plt.xticks(rotation=20)
plt.show()
```



In []: *#Key Summary of Insights*

1. Order Health & Operations: ~92% of orders complete successfully, with cancellations
2. Fulfillment Demand: Order volume peaks on weekends (Friday-Sunday), generating high demand
3. Payment Settlement: A 6.86% transaction leakage rate exists due to failed payments

In []: